EXERCICE 11 · CHANGEMENTS ENTRE LES ÉTAPES

Page Comments avec useFetchApi

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Point de départ** · template (3 fichiers, en entier)
2. **Étape 1 → 2** · correction (3 fichiers)

Départ Point de départ : template

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

Styles fournis (copiés tels quels, pas tapés en live) : src/App.css (74 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg, package.json

DÉPART **src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

DÉPART **src/App.jsx** 12 lignes

```jsx
import "./App.css";
import Comments from "./components/Comments";

function App() {
  return (
    <main className="page">
      <Comments />
    </main>
  );
}

export default App;
```

DÉPART **src/components/Comments.jsx** 41 lignes

```jsx
// Exercice 11 · template : la page Comments s'affiche avec de fausses données, à vous de les remplacer par celles de l'API (les TODO)
// TODO 1 : créer le custom hook useFetchApi(url) dans src/hooks/useFetchApi.js
//          il fait le fetch dans un useEffect et retourne { data, isLoading, isError }
// TODO 2 : importer useFetchApi ici

function Comments() {
  // TODO 3 : remplacer ces fausses données par l'appel au hook (voir l'URL dans l'énoncé)
  const comments = [
    { id: 1, name: "Premier commentaire", email: "alice@mail.com" },
    { id: 2, name: "Deuxième commentaire", email: "bob@mail.com" },
    { id: 3, name: "Troisième commentaire", email: "chloe@mail.com" },
  ];

```

```
  return (
    <section className="card">
      <h2 className="title">Comments</h2>

      {/* TODO 4 : afficher <p className="loading">Loading ...</p> pendant le chargement, et <p
className="error">…</p> si erreur */}
      <table className="table">
        <thead>
          <tr>
            <th>Id</th>
            <th>Comment</th>
            <th>Email</th>
          </tr>
        </thead>
        <tbody>
          {comments.map((comment) => (
            <tr key={comment.id}>
              <td>{comment.id}</td>
              <td>{comment.name}</td>
              <td>{comment.email}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </section>
  );
}

export default Comments;
```

Étape 1 → 2 template → correction

La page Comments branchée sur le hook useFetchApi, avec Loading, l'erreur et le tableau des commentaires.

MODIFIÉ `package.json` +1 −1

```
@@ -1,5 +1,5 @@
  {
-   "name": "exercice-11-template",
+   "name": "exercice-11-correction",
    "private": true,
    "version": "0.0.0",
    "type": "module",
```

MODIFIÉ `src/components/Comments.jsx` +29 −29

```
@@ -1,39 +1,39 @@
- // Exercice 11 · template : la page Comments s'affiche avec de fausses données, à vous de les remplacer par celles de l'API (les TODO)
- // TODO 1 : créer le custom hook useFetchApi(url) dans src/hooks/useFetchApi.js
- //          il fait le fetch dans un useEffect et retourne { data, isLoading, isError }
- // TODO 2 : importer useFetchApi ici
+ import useFetchApi from "../hooks/useFetchApi";

  function Comments() {
-   // TODO 3 : remplacer ces fausses données par l'appel au hook (voir l'URL dans l'énoncé)
-   const comments = [
-     { id: 1, name: "Premier commentaire", email: "alice@mail.com" },
-     { id: 2, name: "Deuxième commentaire", email: "bob@mail.com" },
-     { id: 3, name: "Troisième commentaire", email: "chloe@mail.com" },
-   ];
+   // Toute la logique de fetch est dans le custom hook useFetchApi : la page ne garde que l'affichage
+   const {
+     data: comments,
+     isLoading,
+     isError,
+   } = useFetchApi("https://jsonplaceholder.typicode.com/comments?postId=1");

    return (
      <section className="card">
        <h2 className="title">Comments</h2>
-
-       {/* TODO 4 : afficher <p className="loading">Loading ...</p> pendant le chargement, et <p className="error">…</p> si erreur */}
-       <table className="table">
-         <thead>
-           <tr>
-             <th>Id</th>
-             <th>Comment</th>
-             <th>Email</th>
-           </tr>
-         </thead>
-         <tbody>
-           {comments.map((comment) => (
-             <tr key={comment.id}>
-               <td>{comment.id}</td>
-               <td>{comment.name}</td>
-               <td>{comment.email}</td>
+       {isError && <p className="error">Erreur de chargement</p>}
+       {isLoading ? (
+         <p className="loading">Loading ...</p>
```

```
      ) : (
        <table className="table">
          <thead>
            <tr>
              <th>Id</th>
              <th>Comment</th>
              <th>Email</th>
            </tr>
-           ))}
-         </tbody>
-       </table>
+         </thead>
+         <tbody>
+           {comments.map((comment) => (
+             <tr key={comment.id}>
+               <td>{comment.id}</td>
+               <td>{comment.name}</td>
+               <td>{comment.email}</td>
+             </tr>
+           ))}
+         </tbody>
+       </table>
+     )}
      </section>
    );
  }
```

AJOUTÉ **src/hooks/useFetchApi.js** **+33 −0**

```
@@ -1,0 +1,33 @@
+ import { useEffect, useState } from "react";
+
+ // Custom hook useFetchApi : la logique de fetch (data, loading, error), réutilisable : seule l'URL change
+ function useFetchApi(url) {
+   const [data, setData] = useState([]);
+   const [isLoading, setIsLoading] = useState(false);
+   const [isError, setIsError] = useState(false);
+
+   // useEffect avec [url] : le fetch se relance si l'URL change
+   useEffect(() => {
+     const getData = async () => {
+       setIsLoading(true);
+       try {
+         const response = await fetch(url);
+         // on vérifie la réponse avant de lire le JSON
+         if (!response.ok) {
+           throw new Error("Network response was not ok");
+         }
+         const json = await response.json();
+         setData(json);
+       } catch {
+         setIsError(true);
+       } finally {
+         setIsLoading(false);
+       }
+     };
+     getData();
+   }, [url]);
+
+   return { data, isLoading, isError };
+ }
+
+ export default useFetchApi;
```